

MapLibre GL JS in React + TypeScript

A practical onboarding guide: the mental model, working code for both the React wrapper and the raw API, the pitfalls that cost people days, and a one-week learning path.

01 Where MapLibre stands right now

MapLibre GL JS is the open-source (BSD-licensed) fork of Mapbox GL JS. It renders vector tiles in the browser using WebGL. Two version facts matter before you write any code.

v6 is out and it is a breaking release. MapLibre GL JS v6.0.0 shipped on 22 July 2026. It removed the UMD/CSP browser bundles, the default export, CommonJS support, the internal `map.transform`, and the `MapDataEvent` type. WebGL2 is now mandatory.

In practice: `import maplibregl from 'maplibre-gl'`, which most tutorials and Stack Overflow answers use, no longer works. Use `import * as maplibregl from 'maplibre-gl'` or named imports such as `import { Map, NavigationControl } from 'maplibre-gl'`. `require('maplibre-gl')` throws.

The React wrapper supports v6. `react-map-gl` 8.1.2 added MapLibre v6 support and exports `GlobeControl` from its MapLibre entry point.

First step in the work repo: run `npm ls maplibre-gl react-map-gl`. On v5, older examples work as written. On v6, use the import style in this guide.

02 The mental model

Learn these six concepts and the rest of the API makes sense:

Concept	What it is
Map	The WebGL canvas plus the camera: <code>center</code> , <code>zoom</code> , <code>bearing</code> , <code>pitch</code> .
Style	A JSON document (the “style spec”) listing every source and layer. Your basemap <i>is</i> a style URL.
Source	Where data comes from: <code>geojson</code> , <code>vector</code> (tiles), <code>raster</code> , <code>image</code> .
Layer	How a source is drawn: <code>circle</code> , <code>line</code> , <code>fill</code> , <code>symbol</code> (icons/text), <code>heatmap</code> , <code>fill-extrusion</code> (3D).
Expressions	A small JSON language for data-driven styling, e.g. <code>['get', 'population']</code> .
Events	<code>load</code> , <code>click</code> , <code>mousemove</code> , <code>moveend</code> . Can be scoped to a single layer.

The key difference from Leaflet: you don't add “markers” one by one. You put data in a **source** and describe its appearance with a **layer**, and the GPU draws thousands of features cheaply.

Coordinates are always [longitude, latitude]. That is the opposite of how Google Maps displays them, and it is the single most common bug.

03 Setup

```
npm install maplibre-gl react-map-gl
npm install -D @types/geojson
```

You need a basemap style URL. Options that need no account:

- <https://tiles.openfreemap.org/styles/liberty> — OpenFreeMap, free
- <https://demotiles.maplibre.org/style.json> — very basic demo tiles

Commercial providers (MapTiler, Stadia and others) give you a style URL with an API key. Check which provider the team uses.

04 Your first map (react-map-gl, recommended)

```
// MapView.tsx
import Map, { NavigationControl } from 'react-map-gl/maplibre';
import 'maplibre-gl/dist/maplibre-gl.css'; // forget this and controls/popups look broken

const STYLE_URL = 'https://tiles.openfreemap.org/styles/liberty';

export function MapView() {
  return (
    <Map
      initialViewState={{ longitude: 2.35, latitude: 48.86, zoom: 11 }}
      style={{ width: '100%', height: 500 }} // the map MUST have a height
      mapStyle={STYLE_URL}
    >
      <NavigationControl position="top-right" />
    </Map>
  );
}
```

05 Data, layers, clicks and popups

This is the pattern you will use about 80% of the time: a typed GeoJSON source, a data-driven circle layer, click handling, a popup and an imperative camera move through a ref.

```
import { useCallback, useRef, useState } from 'react';
import Map, {
  Source, Layer, Popup, NavigationControl,
  type LayerProps, type MapLayerMouseEvent, type MapRef,
} from 'react-map-gl/maplibre';
import type { FeatureCollection, Point } from 'geojson';
import 'maplibre-gl/dist/maplibre-gl.css';

type StoreProps = { name: string; revenue: number };

const stores: FeatureCollection<Point, StoreProps> = {
  type: 'FeatureCollection',
```

```

features: [
  { type: 'Feature', geometry: { type: 'Point', coordinates: [2.35, 48.86] },
    properties: { name: 'Central', revenue: 120 } },
  { type: 'Feature', geometry: { type: 'Point', coordinates: [2.30, 48.87] },
    properties: { name: 'West', revenue: 40 } },
],
];

// Define layer styles OUTSIDE the component (or useMemo) so they are stable between renders
const storeLayer: LayerProps = {
  id: 'stores',
  type: 'circle',
  paint: {
    // data-driven radius: 4px at revenue 0, 20px at revenue 200
    'circle-radius': ['interpolate', ['linear'], ['get', 'revenue'], 0, 4, 200, 20],
    'circle-color': ['case', ['>=', ['get', 'revenue'], 100], '#16a34a', '#f97316'],
    'circle-stroke-width': 1,
    'circle-stroke-color': '#fff',
  },
};

type Selected = { lng: number; lat: number; props: StoreProps } | null;

export function StoreMap() {
  const mapRef = useRef<MapRef>(null);
  const [selected, setSelected] = useState<Selected>(null);
  const [cursor, setCursor] = useState('');

  const onClick = useCallback((e: MapLayerMouseEvent) => {
    const f = e.features?.[0];
    if (!f) return setSelected(null);
    const [lng, lat] = (f.geometry as Point).coordinates;
    setSelected({ lng, lat, props: f.properties as StoreProps });
    mapRef.current?.flyTo({ center: [lng, lat], zoom: 14 }); // imperative API via ref
  }, []);

  return (
    <Map
      ref={mapRef}
      initialState={{ longitude: 2.33, latitude: 48.865, zoom: 12 }}
      style={{ width: '100%', height: 500 }}
      mapStyle="https://tiles.openfreemap.org/styles/liberty"
      interactiveLayerIds={['stores']} // only these layers populate e.features
      onClick={onClick}
      onMouseEnter={() => setCursor('pointer')}
      onMouseLeave={() => setCursor('')}
      cursor={cursor}
    >
      <NavigationControl />
      <Source id="stores-src" type="geojson" data={stores}>
        <Layer {...storeLayer} />
      </Source>

      {selected && (
        <Popup longitude={selected.lng} latitude={selected.lat}
          onClose={() => setSelected(null)} closeOnClick={false}>
          <strong>{selected.props.name}</strong>
          <div>Revenue: {selected.props.revenue}k</div>
        </Popup>
      )}
    </Map>
  );
}

```

To update the data, pass a new `data` object to `<Source>` and the map re-renders it. Feature properties come back from click events as plain JSON; nested objects and arrays get stringified, so keep properties flat.

06 Without the wrapper (good to understand)

The codebase may use MapLibre directly. The pattern is a ref plus an effect that creates the map once and tears it down:

```
import { useEffect, useRef } from 'react';
import * as maplibregl from 'maplibre-gl'; // v6-style import
import 'maplibre-gl/dist/maplibre-gl.css';

export function RawMap() {
  const containerRef = useRef<HTMLDivElement>(null);
  const mapRef = useRef<maplibregl.Map | null>(null);

  useEffect(() => {
    if (!containerRef.current) return;
    const map = new maplibregl.Map({
      container: containerRef.current,
      style: 'https://tiles.openfreemap.org/styles/liberty',
      center: [2.35, 48.86],
      zoom: 11,
    });
    map.addControl(new maplibregl.NavigationControl());

    map.on('load', () => {
      map.addSource('stores', { type: 'geojson', data: '/stores.geojson' });
      map.addLayer({ id: 'stores', type: 'circle', source: 'stores',
        paint: { 'circle-radius': 6 } });
      map.on('click', 'stores', (e) => console.log(e.features?.[0]?.properties));
    });

    mapRef.current = map;
    return () => map.remove(); // essential: StrictMode mounts twice in dev
  }, []);

  return <div ref={containerRef} style={{ height: 500 }} />;
}
```

To update data later: `(map.getSource('stores') as maplibregl.GeoJSONSource).setData(newData)`.

07 Expressions cheat sheet

```
['get', 'name'] // read a property
['==', ['get', 'type'], 'shop'] // used in `filter`
['match', ['get', 'type'], 'shop', '#f00', 'cafe', '#0f0', '#999'] // lookup + fallback
['interpolate', ['linear'], ['zoom'], 5, 2, 15, 12] // scale with zoom
['case', ['boolean', ['feature-state', 'hover'], false], '#000', '#888'] // hover
```

08 Gotchas that cost people days

Symptom	Cause and fix
Blank map	The container has no height, or <code>maplibre-gl.css</code> isn't imported.
Point in the ocean	Latitude and longitude swapped. It's always <code>[lng, lat]</code> .
Flicker / re-init	<code>mapStyle</code> , layer objects or <code>initialViewState</code> created inline on every render. Hoist them out of the component or <code>useMemo</code> them.
Slow with many points	Hundreds of <code><Marker></code> components are DOM elements. Use a <code>circle/symbol</code> layer, and <code>cluster: true</code> on the GeoJSON source for large sets.
Data hides labels	Layers draw in the order added. Pass <code>beforeId</code> to slip data under the basemap's labels.
SSR crash (Next.js)	MapLibre needs <code>window</code> . Mark the component <code>'use client'</code> ; load it with <code>dynamic(..., { ssr: false })</code> if needed.
Icons don't show	Custom images must be loaded with <code>map.loadImage</code> + <code>map.addImage</code> before a <code>symbol</code> layer references them.
Broke after upgrade	v6: no default import, no <code>require()</code> , WebGL2 only. Listen to <code>map.on('error', ...)</code> for devices without WebGL2.

09 One-week learning path

- **Day 1** — Get section 04 running. Swap basemap styles. Use `onMove` to log the camera.
 - **Day 2** — Load a real GeoJSON file from the project's domain. Style it with `match` and `interpolate`.
 - **Day 3** — Click popups, hover highlighting with `feature-state`, and `fitBounds` to the data.
 - **Day 4** — Clustering, `symbol` layers with text labels, and a `fill` layer for polygons.
 - **Day 5** — Read the work codebase's map code with this mental model. Find where it creates sources and layers.
-

10 Bookmarks and sources

- MapLibre GL JS examples (copy-pasteable recipes): maplibre.org/maplibre-gl-js/docs/examples
- Style Spec (every paint/layout property and expression): maplibre.org/maplibre-style-spec
- react-map-gl docs: visgl.github.io/react-map-gl

Sources for the version notes

- MapLibre Newsletter, July 2026 — maplibre.org/news/2026-08-02-maplibre-newsletter-jul-2026
- “MapLibre GL JS v6: mandatory WebGL2 and ESM only” — geo.malagis.com
- MapLibre v5 → v6 migration notes — typingmind.com/skills/maplibre-maplibre-v6-migration
- react-map-gl 8.1.2 release notes — via github.com/apache/superset/pull/42831
- react-map-gl repository — github.com/visgl/react-map-gl